



31st March 2026

ProphetMarkets

Smart Contract Security Assessment

SECURITY TEST CONDUCTED BY

FINAL REPORT

FailSafe



Table of Contents

Executive Summary	2
Project Details	3
Scope	3
Deployed Addresses (Polygon Amoy Testnet)	3
Structure and Organization of The Security Report	4
Summary of Findings	5
Finding 1: Operator Privilege Escalation via registerToken	6
Finding 2: Binary Payout Constraint Prevents Market Voiding and Cancellation	8
Finding 3: SafeProxyFactory EIP-712 Signatures Lack Nonce, Deadline, and Dynamic Domain Separator	10
Finding 4: Shared Oracle and Exchange Operator Key	13
Finding 5: Write-Once Resolution With No Dispute or Correction Mechanism	15
Finding 6: Single-Step Ownership Transfer May Brick Oracle Management	17
Appendix: Inherited Risks from Upstream Polymarket Code	19
A-1: Gnosis Safe Signature Bypass via Static CREATE2 Derivation	19
A-2: Integer Truncation-to-Zero Enables Operator Token Extraction	20
A-3: Pre-existing Exchange Balances Captured in matchOrders	20
A-4: Fee Calculation Inconsistency in matchOrders	20
A-5: ERC1155 Callback DoS in matchOrders	21
A-6: No Minimum Admin Count Allows Governance Bricking	21
A-7: Zero takerAmount Orders Bypass Economic Validation	21
A-8: Zero-Amount Fills Corrupt Order State	22
A-9: Admin Factory Address Change Invalidates Pending Orders	22
A-10: Missing Array Length Validation in fillOrders and matchOrders	23

Executive Summary

This report presents the findings of a security assessment conducted on the Prophet Markets smart contract suite, a prediction market platform forking the Polymarket CTF Exchange for deployment on Polygon. The system consists of a thin exchange wrapper (`ProphetCTFExchange`), an AI oracle resolution contract (`Resolution`), a deterministic Gnosis Safe proxy factory (`SafeProxyFactory`), and a modified Safe address derivation library (`PolySafeLib`).

The assessment identified **6 formal findings** within the scoped contracts: 3 High, 2 Medium, and 1 Low. An additional **10 inherited vulnerabilities** from the upstream Polymarket CTF Exchange codebase are documented in the appendix for awareness, as they affect the deployed system even though the underlying code was not modified by Prophet.

The high-severity findings are a Prophet-specific access control weakening in `registerToken` that allows operators to permanently corrupt the token registry, a rigid binary payout constraint in `Resolution.sol` that prevents market voiding or cancellation, and missing replay protections in the `SafeProxyFactory` EIP-712 signature scheme that leave signatures irrevocable, non-expiring, and replayable across chain forks. The medium findings address a deployment configuration where the oracle and exchange operator share a single key and the absence of any dispute or correction mechanism for oracle-reported payouts. A low-severity finding addresses the use of single-step ownership transfer in `Resolution.sol`.

All 6 formal findings have been remediated on the `feat/audit-updates` branches and verified against the cited fix commits. Of the 10 inherited issues, 8 received code-level fixes; A-1 (Gnosis Safe CREATE2 derivation) and A-5 (ERC1155 callback DoS) are Acknowledged. Finding 4 is contract-level resolved with one mainnet deployment check outstanding. Per-finding `Resolution` sections describe the fixes and any operational caveats.

Project Details

Project	Prophet Markets
Repositories	ProphetMarket/contracts, ProphetMarket/ctf-exchange, ProphetMarket/contracts-poly-safe,
Blockchain	Polygon (Amoy Testnet)
Audit Type	Smart Contract Security Assessment
Initial Commits	752f22d (contracts), a83088e (contracts-poly-safe), d1ad800 (ctf-exchange)
Fix Review Commits	968f1b60 (contracts), 3de5d2dd (contracts-poly-safe), bb302fa8 (ctf-exchange)
Timeline	31 March — 28 April 2026

Scope

The following contracts were in scope:

Contract	Lines	Description
ProphetCTFExchange.sol	91	Pragma wrapper around Polymarket exchange mixins with modified access control
Resolution.sol	123	AI oracle payout storage with write-once semantics and ownership controls
PolySafeProxyFactory.sol	109	Deterministic Gnosis Safe proxy deployer via CREATE2 and EIP-712 signatures
PolySafeLib.sol	38	Safe address derivation library with updated proxy creation bytecode

Deployed Addresses (Polygon Amoy Testnet)

Contract	Address
ProphetCTFExchange	0x009dbB7c0DFf41F33190887eC4be076c9ad92df9
Resolution	0x3747c131432EfFe7658EFfe159AF95454037dF118
SafeProxyFactory	0x415bFDf7cD6A60f070c6fCD206B67c0cFcf0B623

Structure and Organization of The Security Report

Issues are tagged as “Open”, “Acknowledged”, “Partially Resolved”, “Resolved” or “Closed” depending on whether they have been fixed or addressed.

- Open: The issue has been reported and is awaiting remediation from developer team.
- Acknowledged: The developer team has reviewed and accepted the issue but has decided not to fix it.
- Partially Resolved: Mitigations have been applied, yet some risks or gaps still remain.
- Resolved: The issue has been fully addressed and no further work is necessary.
- Closed: The issue is deemed no longer pertinent or actionable.

Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

✖ Critical	The issue affects the platform in such a way that funds may be lost, allocated incorrectly, or otherwise result in a significant loss.
⚠ High	The issue affects the ability of the platform to compile or operate in a significant way.
⚠ Medium	The issue affects the ability of the platform to operate in a way that doesn't significantly hinder its behavior.
🟢 Low	The issue has minimal impact on the platform's ability to operate.
ℹ Info	The issue is informational in nature and does not pose any direct risk to the platform's operation.

Summary of Findings

Severity	Total	Open	Acknowledged	Partially Resolved	Resolved	Closed
🔴 High	3	-	-	-	3	-
🟡 Medium	2	-	-	-	2	-
🟢 Low	1	-	-	-	1	-
Total	6	-	-	-	6	-

Finding 1: Operator Privilege Escalation via registerToken

Severity: HIGH

Status: Resolved

Description

ProphetCTFExchange.registerToken allows both admins and operators to register token pairs, a deliberate weakening from the upstream Polymarket CTFExchange which restricts this function to onlyAdmin. The operator role is intended for order settlement — a semi-trusted hot wallet — not for market configuration. Combined with the write-once semantics of Registry._registerToken (which reverts with AlreadyRegistered on re-registration) and the absence of any unregister mechanism, a compromised operator can permanently corrupt the token registry by registering tokens with incorrect conditionId values or mismatched complements.

Once a token pair is poisoned, the correct registration is blocked forever. Any matchOrders MINT or MERGE operations for the affected pair will interact with the wrong CTF condition, either reverting (DoS) or producing incorrect outcome tokens. Recovery requires redeploying the entire exchange contract.

Source

ProphetCTFExchange.sol - registerToken (Modified Access Control):

contracts/src/ProphetCTFExchange.sol:87-90 - Allows operators:

```
1 function registerToken(uint256 token, uint256 complement, bytes32 conditionId) external {
2     if (admins[msg.sender] != 1 && operators[msg.sender] != 1) revert NotAdminOrOperator();
3     _registerToken(token, complement, conditionId);
4 }
```

CTFExchange.sol - registerToken (Original, Admin-Only):

```
1 function registerToken(uint256 token, uint256 complement, bytes32 conditionId) external onlyAdmin {
2     _registerToken(token, complement, conditionId);
3 }
```

Registry.sol - _registerToken (Write-Once):

ctf-exchange/src/exchange/mixins/Registry.sol:38-49:

```

1  function _registerToken(uint256 token0, uint256 token1, bytes32 conditionId) internal {
2      if (token0 == token1 || (token0 == 0 || token1 == 0)) revert InvalidTokenId();
3      if (registry[token0].complement != 0 || registry[token1].complement != 0)
4          revert AlreadyRegistered();
5      registry[token0] = OutcomeToken({complement: token1, conditionId: conditionId});
6      registry[token1] = OutcomeToken({complement: token0, conditionId: conditionId});
7      emit TokenRegistered(token0, token1, conditionId);
8      emit TokenRegistered(token1, token0, conditionId);
9  }

```

No on-chain validation exists to verify that the `conditionId` is prepared in the CTF contract or that the token IDs correspond to it.

Deployed Configuration (Verified):

Operator `0x65E1510A914792d0Caa51DC728B5f83FeEdA4329` is registered on the exchange at `0x009dbB7c0DFf41F33190887eC4be076c9ad92df9` and can call `registerToken`.

Impact

- A compromised operator hot wallet can permanently brick specific markets in a single transaction
- Correct token registration is blocked forever for poisoned token IDs — no `unregister` function exists
- `matchOrders` MINT/MERGE operations route through the wrong `conditionId`, causing either reverts or interaction with incorrect CTF conditions
- The only recovery path is full exchange redeployment, which invalidates all existing orders and approvals
- The operator can front-run legitimate admin token registrations for upcoming markets

Remediation

Revert the access control to `onlyAdmin`, matching the upstream Polymarket implementation:

```

1  function registerToken(uint256 token, uint256 complement, bytes32 conditionId) external onlyAdmin {
2      _registerToken(token, complement, conditionId);
3  }

```

If operators must participate in registration, implement a two-step propose-and-confirm pattern where the operator proposes and an admin confirms.

Finding 2: Binary Payout Constraint Prevents Market Voiding and Cancellation

Severity: HIGH

Status: Resolved

Description

`Resolution._validPayouts` only accepts `[1,0]` or `[0,1]` payout vectors, rejecting the `[1,1]` payout that the Gnosis Conditional Token Framework uses to void or cancel a market. When a real-world event is cancelled, tied, or deemed invalid, the oracle has no mechanism to trigger a proportional refund. Either the oracle refuses to resolve the market (permanently locking all collateral in the CTF) or it is forced to pick an arbitrary winner (causing total loss for the other side).

Event cancellations are routine in prediction markets — postponed sports events, withdrawn political candidates, ambiguous outcomes. The protocol has no emergency bypass, admin override, or alternative cancellation path.

Source

****Resolution.sol - _validPayouts:****

contracts/src/Resolution.sol:120-122:

```
1 function _validPayouts(uint256 a, uint256 b) private pure returns (bool) {
2     return (a == 1 && b == 0) || (a == 0 && b == 1);
3 }
```

Resolution.sol - reportPayouts:

contracts/src/Resolution.sol:63-76:

```
1 function reportPayouts(bytes32 conditionId, uint256[] calldata payouts, string calldata ipfsCid)
2     external
3     onlyOracle
4     {
5         if (_reported[conditionId]) revert AlreadyReported(conditionId);
6         if (payouts.length != 2) revert InvalidPayoutsLength(payouts.length);
7         if (!_validPayouts(payouts[0], payouts[1])) revert InvalidPayoutValues();
8
9         _reported[conditionId] = true;
10        _payouts[conditionId] = payouts;
11        _ipfsCids[conditionId] = ipfsCid;
12
13        emit PayoutsReported(conditionId, payouts, ipfsCid);
14    }
```

The Gnosis CTF `reportPayouts` accepts arbitrary payout vectors including `[1,1]`, which splits collateral equally among all position holders. Prophet's validation layer blocks this.

Impact

- Cancelled or invalidated events have no resolution path — collateral is permanently locked in the CTF contract
- If the oracle picks an arbitrary winner to unblock funds, the losing side suffers 100% loss on a non-event
- Write-once semantics (`_reported[conditionId] = true`) prevent correction after the fact
- Financial impact scales with total market volume for the affected condition

Remediation

Extend `_validPayouts` to accept `[1,1]` for market cancellation:

```
1 function _validPayouts(uint256 a, uint256 b) private pure returns (bool) {  
2     return (a == 1 && b == 0) || (a == 0 && b == 1) || (a == 1 && b == 1);  
3 }
```

Optionally, add a dedicated `voidMarket(bytes32 conditionId, string calldata ipfsCid)` function restricted to the owner or oracle that reports `[1,1]` payouts.

Finding 3: SafeProxyFactory EIP-712 Signatures Lack Nonce, Deadline, and Dynamic Domain Separator

Severity: HIGH

Status: Resolved

Description

The `SafeProxyFactory.createProxy` function authorizes Safe deployment via an EIP-712 signature over `CreateProxy(address paymentToken, uint256 payment, address paymentReceiver)`. This signed struct omits three standard protections:

1. **No nonce:** Once a signature is issued, there is no on-chain mechanism to revoke it. The signer cannot cancel a pending deployment authorization. CREATE2 prevents same-address replay, but if a user signs multiple signatures with different payment parameters, all remain valid indefinitely and anyone can choose which to execute.
2. **No deadline:** Signatures never expire. A signature issued months or years ago can be executed at any time.
3. **Static domainSeparator:** The `domainSeparator` is computed once in the constructor and stored as a state variable. Unlike the ERC20 implementation in the same codebase (which checks `block.chainid` at runtime and recomputes when it changes), the factory uses the stale value. On a chain fork, signatures are valid on both chains.

Source

PolySafeProxyFactory.sol - CREATE_PROXY_TYPEHASH:

contracts-poly-safe/src/PolySafeProxyFactory.sol:27-28:

```
1 bytes32 public constant CREATE_PROXY_TYPEHASH =  
2     keccak256("CreateProxy(address paymentToken,uint256 payment,address paymentReceiver)");
```

No nonce or deadline fields.

PolySafeProxyFactory.sol - Constructor (Static domainSeparator):

contracts-poly-safe/src/PolySafeProxyFactory.sol:42-47:

```

1  constructor(address _masterCopy, address _fallbackHandler) {
2      masterCopy = _masterCopy;
3      fallbackHandler = _fallbackHandler;
4      domainSeparator =
5          keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(NAME)),
6              _getChainIdInternal(), address(this)));
7  }

```

Computed once, never recomputed.

****PolySafeProxyFactory.sol - _getSigner (Uses Stale Separator):****

contracts-poly-safe/src/PolySafeProxyFactory.sol:93-101:

```

1  function _getSigner(...) internal view returns (address) {
2      bytes32 structHash = keccak256(abi.encode(
3          CREATE_PROXY_TYPEHASH, paymentToken, payment, paymentReceiver));
4      bytes32 digest = keccak256(abi.encodePacked(
5          "\x19\x01", domainSeparator, structHash));
6      return ECDSA.recover(digest, sig.v, sig.r, sig.s);
7  }

```

No nonce check, no deadline validation. Uses static domainSeparator.

PolySafeProxyFactory.sol - createProxy (Payment Drain Vector):

contracts-poly-safe/src/PolySafeProxyFactory.sol:70-91:

```

1  function createProxy(...) external {
2      address owner = _getSigner(paymentToken, payment, paymentReceiver, createSig);
3      // ... CREATE2 deployment ...
4      proxy.setup(owners, 1, address(0), "", fallbackHandler,
5          paymentToken, payment, paymentReceiver);
6  }

```

The proxy.setup call passes paymentToken, payment, and paymentReceiver directly to GnosisSafe.setup, which transfers payment tokens from the newly deployed Safe during initialization. If the deterministic Safe address is pre-funded, these funds are drained atomically during deployment.

Impact

- Signatures cannot be revoked after issuance — permanent authorization until consumed by CREATE2
- A user who signs multiple signatures with different payment amounts cannot invalidate the higher-fee version
- On chain forks, createProxy signatures can be replayed to deploy Safes on the fork chain
- If a counterfactual Safe address is pre-funded and the signature includes non-zero payment, anyone with the signature can trigger deployment and drain the pre-funded balance

Remediation

1. Add `nonce` and `deadline` fields to `CREATE_PROXY_TYPEHASH`:

```
1 bytes32 public constant CREATE_PROXY_TYPEHASH = keccak256(  
2     "CreateProxy(address paymentToken,uint256 payment,address paymentReceiver,uint256 nonce,uint256 deadline)"  
3 );
```

2. Maintain a per-signer nonce mapping and check `require(block.timestamp <= deadline)` in `createProxy`
3. Recompute `domainSeparator` dynamically when `block.chainid` differs from the cached value

Finding 4: Shared Oracle and Exchange Operator Key

Severity: MEDIUM

Status: Resolved

Description

The Resolution oracle and the ProphetCTFExchange operator are configured to the same address (0x65E1510A914792d0Caa51DC728B5f83FeEdA4329). This creates a single point of compromise where one key controls both market resolution (determining which outcome wins) and order execution (filling trades). A compromised key could resolve a market in a specific direction and simultaneously fill orders that profit from that resolution, all within a single block.

Source

On-chain verification:

Resolution oracle:

```
1 cast call 0x3747c131432EfFe7658EF159AF95454037dF118 "oracle()" --rpc-url https://rpc-amoy.polygon.technology→
2 0x65E1510A914792d0Caa51DC728B5f83FeEdA4329
```

Exchange operator:

```
1 cast call 0x009dbB7c0DFf41F33190887eC4be076c9ad92df9 "operators(address)"
2 0x65E1510A914792d0Caa51DC728B5f83FeEdA4329 --rpc-url https://rpc-amoy.polygon.technology→
3 1
```

Impact

- Single key compromise enables simultaneous market manipulation and order execution
- The oracle can front-run its own resolution by filling profitable orders before reporting payouts
- Users trust the operator to execute fair trades and the oracle to report honest outcomes — combining these roles into one key eliminates the separation of duties that protects against insider threats

Remediation

Use separate keys for the oracle and operator roles. Consider using a multisig or timelock for the oracle role, since market resolution is a high-value, infrequent operation that benefits from additional security.

Resolution

Resolved at the contract level; pending deployment-time verification at mainnet. A three-role architecture has been wired in: `ProphetCTFExchange.setOracle()` (admin-gated) lets the admin designate an oracle address distinct from the operator, and the deploy scripts (`Deploy.s.sol`, `SetOracle.s.sol`) automate the role assignment. The Resolution oracle and the exchange operator are now logically separate roles.

The current Amoy testnet deployment still uses a single key (`0x65E1510A914792d0Caa51DC728B5f83FeEdA4329`) for both Operator and Oracle roles for development convenience. We'd recommend confirming at mainnet that `Resolution.oracle()` and the exchange operator resolve to distinct keys, per Prophet's plan to use dedicated Privy Server Wallets.

Fix commits (contracts/): `25f0fdf`, `37d9849`, `bc1aa94`.

Finding 5: Write-Once Resolution With No Dispute or Correction Mechanism

Severity: MEDIUM

Status: Resolved

Description

`Resolution.reportPayouts` enforces strict write-once immutability: once payouts are stored for a `conditionId`, no function in the contract can modify or clear them. No timelock, dispute period, or admin override exists. Combined with the use of an AI oracle, this means a single incorrect report permanently misallocates all collateral in the affected market with no recovery path.

The contract has only three state-changing external functions: `reportPayouts` (oracle-gated, write-once), `setOracle` (owner-gated, rotates oracle address), and `renounceOwnership` (reverts). None can alter previously stored payouts or reset the `_reported` flag.

Source

Resolution.sol - Write-once guard:

contracts/src/Resolution.sol:67:

```
1 if (_reported[conditionId]) revert AlreadyReported(conditionId);
```

No function in the contract sets `_reported[conditionId]` back to `false` or modifies `_payouts[conditionId]` after initial storage.

Resolution.sol - Complete state-changing surface:

- `reportPayouts` — write-once, oracle only
- `setOracle` — rotates oracle address, cannot fix past reports
- `renounceOwnership` — always reverts
- `transferOwnership` — inherited from `Ownable`, changes owner only

Impact

- A single incorrect AI oracle report permanently locks 100% of collateral in the wrong outcome
- AI oracle error is non-trivial: hallucinations, stale data, prompt injection, or key compromise
- No dispute mechanism for affected users to challenge an incorrect resolution
- No admin emergency override to clear or correct a report
- Impact scales with market size and is irreversible

Remediation

Implement a time-delayed finalization pattern:

1. `reportPayouts` stores payouts in a pending state with a timestamp
2. A dispute window (e.g., 24–48 hours) allows the owner or a designated dispute resolver to challenge the report
3. After the window expires without challenge, a `finalizePayouts` call commits the result

Alternatively, add an owner-gated `resetReport(bytes32 conditionId)` emergency function behind a timelock.

Finding 6: Single-Step Ownership Transfer May Brick Oracle Management

Severity: LOW

Status: Resolved

Description

Resolution inherits OpenZeppelin Ownable (single-step transferOwnership) rather than Ownable2Step. The contract explicitly acknowledges that losing ownership is catastrophic by disabling renounceOwnership:

```
1 /// @notice Disabled – renouncing ownership would permanently brick oracle management.
2 function renounceOwnership() public pure override {
3     revert RenounceDisabled();
4 }
```

However, transferOwnership(address) remains as a single-step operation. If the owner accidentally transfers to an incorrect address (typo, wrong-chain address, non-receiving contract), ownership is immediately and irrevocably lost. Since setOracle is onlyOwner, the oracle can never be rotated again — all unresolved markets depend on the current oracle key indefinitely.

Ownable2Step is available in the same OpenZeppelin v5 dependency and requires the new owner to call acceptOwnership(), confirming the address is controlled.

Source

Resolution.sol - Import and inheritance:

contracts/src/Resolution.sol:4,9:

```
1 import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
2 ...
3 contract Resolution is Ownable {
```

Uses Ownable, not Ownable2Step.

Impact

- Accidental ownership transfer permanently bricks setOracle, making the oracle non-rotatable
- If the oracle key is later compromised or lost, all unresolved markets are permanently bricked
- The contract's own documentation acknowledges this class of risk but only guards against renounceOwnership

Remediation

Replace the import and inheritance with Ownable2Step:

```
1 import {Ownable2Step, Ownable} from "@openzeppelin/contracts/access/Ownable2Step.sol";  
2 ...  
3 contract Resolution is Ownable2Step {
```

Appendix: Inherited Risks from Upstream Polymarket Code

The following issues exist in the Polymarket CTF Exchange mixin code that `ProphetCTFExchange` inherits. These are **not Prophet-authored code** and fall outside the formal audit scope, but they affect the deployed system. They are documented here for awareness and risk management.

A-1: Gnosis Safe Signature Bypass via Static CREATE2 Derivation

Inherited Severity: HIGH

Status: Acknowledged

`verifyPolySafeSignature` in `Signatures.sol` verifies Gnosis Safe orders by checking if the signer's EOA deterministically maps to the Safe address via CREATE2. It never queries the Safe's current on-chain owner set via `isOwner()` or EIP-1271 `isValidSignature`. If a user rotates their Safe's owners after a key compromise, the removed key can still sign valid exchange orders because the CREATE2 derivation is immutable. The `POLY_1271` signature type correctly delegates to the Safe's own signature validation, proving the omission in `POLY_GNOSIS_SAFE` is a defect.

Resolution

Acknowledged. Prophet's stated rationale is that `POLY_GNOSIS_SAFE` relies on deterministic CREATE2 address derivation by design, and migrating to EIP-1271 `isValidSignature` would require a fundamental redesign of the signature verification path. Compensating mitigations:

- Prophet controls all Safe deployments through its own factory and operator
- Safe owner rotation is not exposed to users in the current product scope
- A deployment-time bytecode assertion (commit `9b0205f` in `contracts/`) ensures the CREATE2 derivation matches the live factory at deploy time

The residual risk — sophisticated users modifying their Safe's owner set or fallback handler out-of-band — is accepted under the current product scope.

A-2: Integer Truncation-to-Zero Enables Operator Token Extraction

Inherited Severity: HIGH

Status: Resolved

`CalculatorHelper.calculateTakingAmount` performs `makingAmount * takerAmount / makerAmount`. When `makingAmount` is small relative to `makerAmount / takerAmount`, integer division truncates to 0. An operator can fill orders with tiny amounts where each fill transfers maker tokens (`making > 0`) but the payment truncates to 0. No `taking > 0` check exists anywhere in the pipeline. For SELL orders at price < 1.0 (common in prediction markets), `fillAmount=1` produces `taking=0`, and the operator can batch thousands of these via `fillOrders`.

A-3: Pre-existing Exchange Balances Captured in matchOrders

Inherited Severity: MEDIUM

Status: Resolved

`_updateTakingWithSurplus` reads the entire exchange balance of `takerAssetId` via `_getBalance` (which returns `IERC20.balanceOf(address(this))`), not a delta from a pre-call snapshot. Any tokens present in the exchange before a `matchOrders` call — from accidental transfers, rounding dust, or deliberate deposits — are swept to the operator-chosen taker. The refund mechanism at the end of `_matchOrders` exhibits the same behavior for `makerAssetId`.

A-4: Fee Calculation Inconsistency in matchOrders

Inherited Severity: MEDIUM

Status: Resolved

In `_fillOrder`, `calculateFee` receives `order.makerAmount` and `order.takerAmount` for price derivation, matching the user's signed terms. In `_matchOrders`, `calculateFee` receives the actual `making` and surplus-adjusted `taking`, changing the effective price. When surplus exists from favorable maker fills, the taker's fee can increase significantly (100%+ in some cases). The operator, who receives fees, controls which maker fills generate surplus.

A-5: ERC1155 Callback DoS in matchOrders

Inherited Severity: MEDIUM

Status: Acknowledged

`cancelOrder` and `incrementNonce` lack the `nonReentrant` modifier. During `matchOrders`, ERC1155 `safeTransferFrom` triggers `onERC1155Received` on maker contracts. A malicious maker contract can call `cancelOrder` on its own later orders in the batch during this callback, causing the entire `matchOrders` transaction to revert when the loop reaches the cancelled order.

Resolution

Acknowledged. Prophet's stated rationale: in their architecture, the operator controls order matching and can exclude malicious maker contracts from batch submissions; users interact through Gnosis Safe wallets in the default Prophet-deployed configuration, which limits the attack surface. They will revisit this if the trading model evolves to allow contract-based makers.

A-6: No Minimum Admin Count Allows Governance Bricking

Inherited Severity: MEDIUM

Status: Resolved

`Auth.sol` allows the last admin to remove themselves via `removeAdmin(self)` or `renounceAdminRole()` without checking if any admin remains. Since `addAdmin` requires `onlyAdmin`, once zero admins exist, all admin-gated functions (pause, unpause, role management, factory updates, token registration) are permanently inaccessible. The `Resolution.sol` contract explicitly prevents this pattern by disabling `renounceOwnership()`, demonstrating awareness of this class of bug.

A-7: Zero takerAmount Orders Bypass Economic Validation

Inherited Severity: MEDIUM

Status: Resolved

Orders with `takerAmount=0` pass all validation in `_validateOrder`. `isCrossing` returns **true** unconditionally for zero-takerAmount orders, `calculateTakingAmount` returns 0, and `calculateFee` returns 0 (`price=0` fails the `price > 0` check). The operator can fill such orders, extracting the maker's entire `makerAmount` for zero payment. This requires a user to sign a malformed order, but the contract provides no defense-in-depth against this degenerate case.

A-8: Zero-Amount Fills Corrupt Order State

Inherited Severity: LOW

Status: Acknowledged

`fillOrder` with `fillAmount=0` succeeds: `_updateOrderStatus` initializes `remaining` to `makerAmount` for new orders then subtracts 0, writing to storage without advancing the fill. `OrderFilled` events emit with zero amounts. No direct fund loss, but causes unnecessary storage writes and misleading events.

Resolution

Acknowledged. The operator controls `fillAmount` end-to-end and is not expected to submit zero-amount fills under normal operation. In practice, the impact is more likely to surface as extra storage writes and event-log noise than as direct fund loss, though we'd still suggest a `making > 0` guard as cheap defense-in-depth if revisited.

A-9: Admin Factory Address Change Invalidates Pending Orders

Inherited Severity: LOW

Status: Resolved

Calling `setProxyFactory` or `setSafeFactory` immediately changes the factory address used in signature verification for `POLY_PROXY` and `POLY_GNOSIS_SAFE` order types. All pending orders using these types become instantly unfillable because the computed wallet addresses no longer match. No migration path, grace period, or dual-verification window exists. Note: `proxyFactory` is set to `address(0)` in the current deployment, so only `POLY_GNOSIS_SAFE` orders via `setSafeFactory` are affected.

Resolution

Resolved via runbook (operational mitigation). No code-level dual-verification window was added. Factory address changes are now documented as admin operations requiring an off-chain trading pause prior to invocation, so no in-flight orders are silently invalidated. Reasonable for a LOW-severity issue given the operator-controlled order submission flow.

A-10: Missing Array Length Validation in `fillOrders` and `matchOrders`

Inherited Severity: INFO

Status: Resolved

`fillOrders` iterates `orders.length` but accesses `fillAmounts[i]` without verifying `orders.length == fillAmounts.length`. Similarly, `_fillMakerOrders` iterates `makerOrders.length` but accesses `makerFillAmounts[i]`. Mismatched lengths produce opaque EVM out-of-bounds reverts rather than descriptive error messages.